

งานปฏิบัติการกลุ่ม — การออกแบบและวิเคราะห์อัลกอริทึมด้วย Stack (ภาษา Java)

Emergency Workflow

Event Stack และ State Machine

กลุ่มที่ 5 | Algorithm A: Event Stack Method vs Algorithm B: Event Stack + State Machine

สมาชิกกลุ่ม

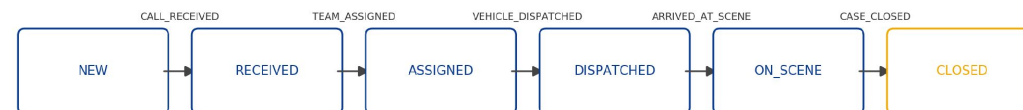
1. สิทธิโชค ดีเหลือ 66122250018
2. ภัทรกร อนุกุล 68122250068
3. ระพีพัฒน์ สระทองอ่อน 68122250065
4. สุทธาทิพย์ เพ็ชรแดง 66122250085
- 5.
- 6.

CASE STORY

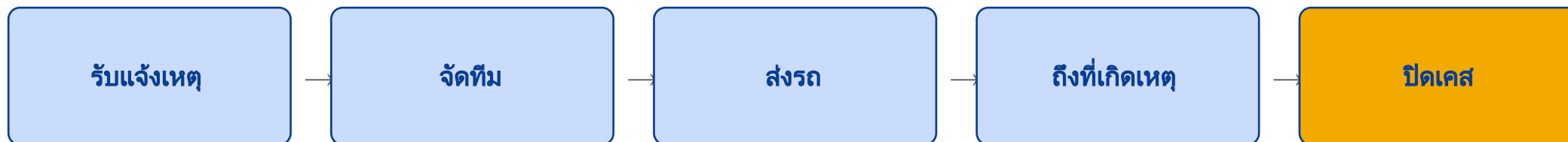
ศูนย์รับแจ้งเหตุฉุกเฉิน : ลำดับ Action ที่ต้องถูกควบคุมอย่างเคร่งครัด

ปัญหา: ศูนย์รับแจ้งเหตุฉุกเฉินต้องบันทึกลำดับ Action ของแต่ละเคสให้ถูกต้องเสมอ ป้องกันการปิดเคสก่อนถึงที่เกิดเหตุ หรือข้ามขั้นตอนสำคัญใด ๆ

ความต้องการ : ระบบต้องรองรับ Undo และ Redo เพื่อแก้ไขข้อผิดพลาดของเจ้าหน้าที่ได้ทันที และต้องล้าง Redo History เมื่อมีการเพิ่ม Action ใหม่หลัง Undo บางส่วน



กฎสำคัญ : ห้ามเพิ่ม Action ที่ไม่ตรงกับ currentState ตาม Transition Table และเมื่อเพิ่ม Action ใหม่หลัง Redo บางส่วน ต้องล้าง Redo Stack ทั้งหมดทันที



NEW → RECEIVED → ASSIGNED → DISPATCHED → ON_SCENE → CLOSED

Input, Output และ Constraints

Input

- คำสั่ง ADD <ActionType> / UNDO / REDO
- ActionType 5 ค่าตามลำดับ Workflow
- สถานะปัจจุบันของ Event Stack / Redo Stack

Output

- boolean สำเร็จ/ถูกปฏิเสธของ ADD
- currentState หลังทำทุกคำสั่ง
- Event History และ Redo History

Constraints

- Workflow เป็นเส้นตรง ไม่มีการแตกสาขา
- CLOSED เป็น Terminal State
- เพิ่ม Action ใหม่หลัง Redo ต้องล้าง Redo Stack

Algorithm A — Event Stack Method

แนวคิดหลัก : ไม่เก็บ currentState แยกต่างหาก ทุกครั้งที่ต้องตรวจสอบ Action ใหม่หรืออ่านสถานะ จะสแกน Event Stack ทั้งหมดผ่าน Transition Table เพื่อคำนวณสถานะใหม่ (Derived State)

ข้อดี

Single Source of Truth เดียว ไม่มี State ซ้ำซ้อนที่อาจไม่ตรงกัน

ข้อจำกัด

addAction() และ getCurrentState() ต้องสแกนทั้ง Stack ทุกครั้ง

Time Complexity

$O(n)$

addAction() / getCurrentState()

สแกน Event Stack ทั้งหมด n รายการ

undo() / redo(): $O(1)$

Auxiliary Space: $O(n)$ (Temporary Stack)

Pseudocode (ย่อ): DeriveCurrentState $\rightarrow$ Pop ทั้งหมดไป Temp $\rightarrow$ ไล่ผ่าน TransitionTable $\rightarrow$ Push กลับ

Algorithm B — Event Stack ร่วมกับ State Machine

Time Complexity

$O(1)$

addAction() / getCurrentState()

อ่าน/เขียนตัวแปร currentState แบบ Cache

undo() / redo(): $O(1)$

Auxiliary Space: $O(1)$ เพิ่มเติม

Pseudocode (ย่อ): nextState <-
TransitionTable.getNextState(currentState, action) → Push
→ currentState <- nextState

แนวคิดหลัก : เก็บ currentState เป็นตัวแปร Cache ตัวเดียว ทุกครั้งที่เพิ่ม Action หรือ Undo/Redo จะอัปเดต currentState ทันทีแบบ Incremental โดยไม่ต้องสแกน Stack ทั้งหมด

ข้อดี

ตรวจสอบและอ่านสถานะได้ $O(1)$ เหมาะกับระบบที่มี Event จำนวนมาก

ข้อจำกัด

ต้องดูแล currentState ให้สอดคล้องกับ Stack เสมอ (Invariant)

ตัวอย่างการทำงาน

Step-by-step: สถานการณ์บังคับ

ตัวอย่างการทำงานแบบ Step-by-step: สถานการณ์บังคับ (ADD x5 -> UNDO x2 -> REDO -> ADD ใหม่)



วิธีอ่านภาพ : กล่องสีทึบ = รายการบนสุด (Top) ของแต่ละ Stack | กรอบเส้นประสีทอง = รายการที่เพิ่งถูก Push/Pop ในขั้นตอนนั้น | ลูกศรโค้ง = ทิศทางการย้ายข้อมูลระหว่าง Event Stack กับ Redo Stack ตอน Undo/Redo

ลำดับเหตุการณ์ทั้ง 9 ขั้นตอน

1-5) ADD ตามลำดับ Workflow ที่ถูกต้อง — Push เข้า Event Stack ทีละรายการจนถึง CASE_CLOSED

6-7) UNDO ×2 — Pop รายการบนสุดออกจาก Event Stack แล้ว Push เข้า Redo Stack (CASE_CLOSED แล้ว ARRIVED_AT_SCENE)

8) REDO — Pop CASE_CLOSED ออกจาก Redo Stack กลับเข้า Event Stack, currentState กลับเป็น ON_SCENE

9) ADD ใหม่ (CASE_CLOSED) — Push สำเร็จ พร้อมล้าง Redo Stack ทั้งหมดตามกฎของระบบ

Correctness Argument — Loop Invariant (Algorithm B)

Invariant: หลังทุก Operation (ADD/UNDO/REDO) currentState ต้องเท่ากับผลของการไล่ Transition Table ตาม Event Stack เสมอ และลำดับ Action ใน Stack ต้องเป็น Workflow ที่ถูกต้อง

Initialization

ก่อนเริ่ม Event Stack ว่าง และ currentState = NEW ตรงกับผลไล่ Transition Table จาก Stack ว่าง → Invariant จริงตั้งแต่ต้น

Maintenance

addAction สำเร็จจะ Push + อัปเดต currentState พร้อมกันเสมอ ถ้าล้มเหลวจะไม่แก้ไขอะไรเลย ; undo/redo ย้อน/ทำ Transition เดียวกับที่ Push เข้ามา จึงรักษาความสอดคล้องเสมอ

Termination

ทุก Operation จบในเวลาจำกัดเสมอ (ไม่มี Loop ขึ้นกับ n) เมื่อหยุดทำงาน currentState ยังตรงกับ Event Stack เสมอ — ยืนยันด้วย ผล TestCases 20/20 PASS

Big-O Analysis — วิเคราะห์ทุกฟังก์ชันของระบบ

ฟังก์ชัน	Algorithm A	Algorithm B
addAction()	$O(n)$	$O(1)$
undo() / redo()	$O(1)$	$O(1)$
getCurrentState()	$O(n)$	$O(1)$
getEventHistory() / displayStatus()	$O(n)$	$O(n)$
Auxiliary Space	$O(n)$ (Temporary Stack)	$O(1)$
Total Space	$O(n)$	$O(n)$

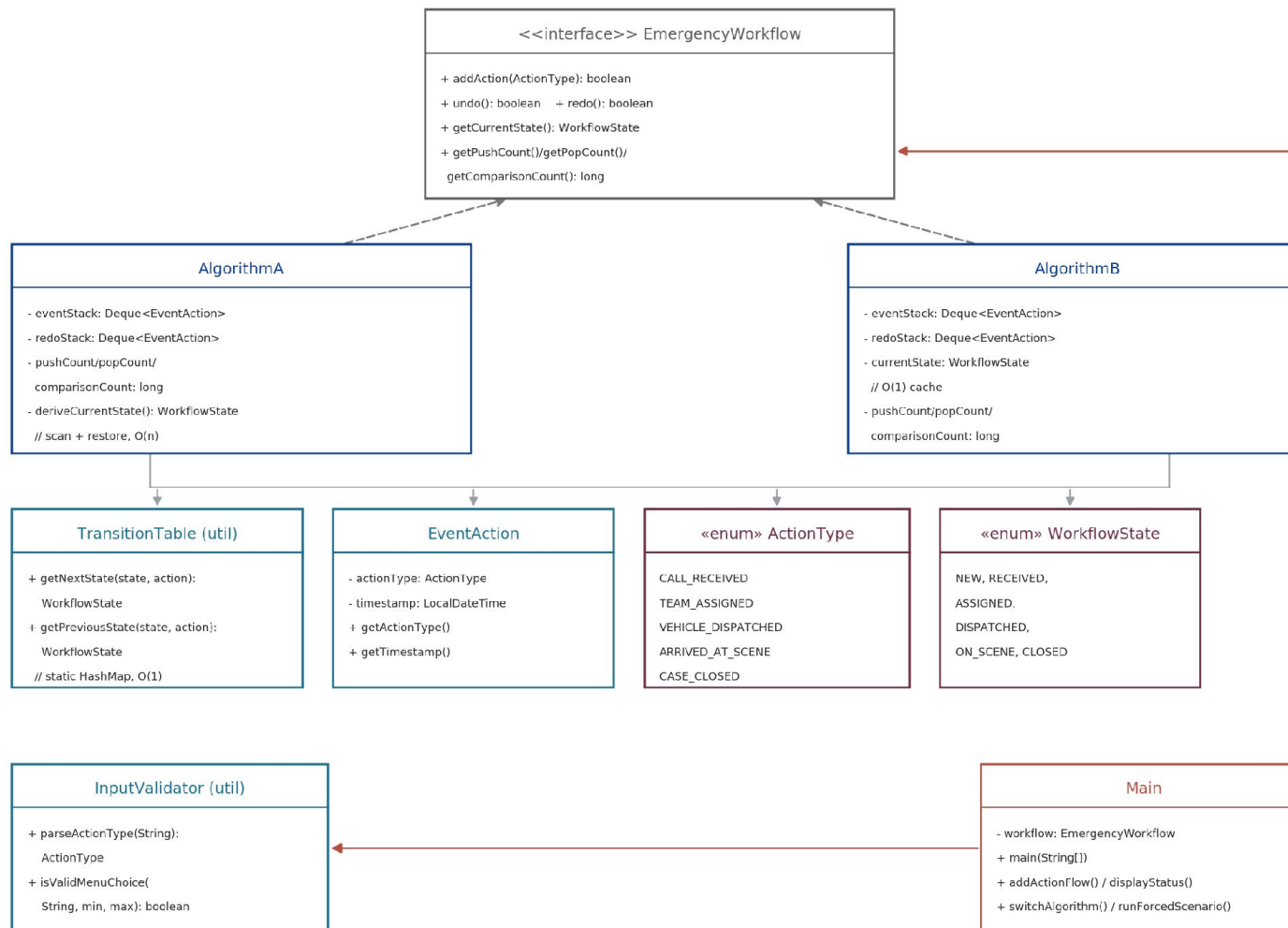
สรุป

Algorithm A ต้อง $O(n)$ ทุกครั้งที่ addAction หรืออ่านสถานะ เพราะคำนวณจากข้อมูล
ต้นทางเสมอ

Algorithm B ใช้ $O(1)$ คงที่ เพราะมีตัวแปร
Cache ที่อัปเดตร่วมกับ State Machine

ยิ่ง n ใหญ่ ผลต่างยิ่งชัดเจน — สอดคล้องกับ
ผลทดลองจริง

Class Diagram และโครงสร้างโปรแกรม



TEST CASES

ชุดทดสอบ 10 กรณี ครอบคลุม Normal / Edge / Error Case

20 / 20

PASS

10 Test Cases × 2 Algorithms
รันจริงผ่าน TestCases.java บน
OpenJDK 21

✓ TC01 Workflow ถูกต้องตั้งแต่ต้นจนจบ

✓ TC02 เพิ่ม Action ผิดลำดับ

✓ TC03 ปิด Case ก่อนถึงที่เกิดเหตุ

✓ TC04 Undo หนึ่งครั้ง

✓ TC05 Undo หลายครั้ง

✓ TC06 Redo บางส่วน

✓ TC07 Add ใหม่หลัง Redo บางส่วน ล้าง Redo Stack

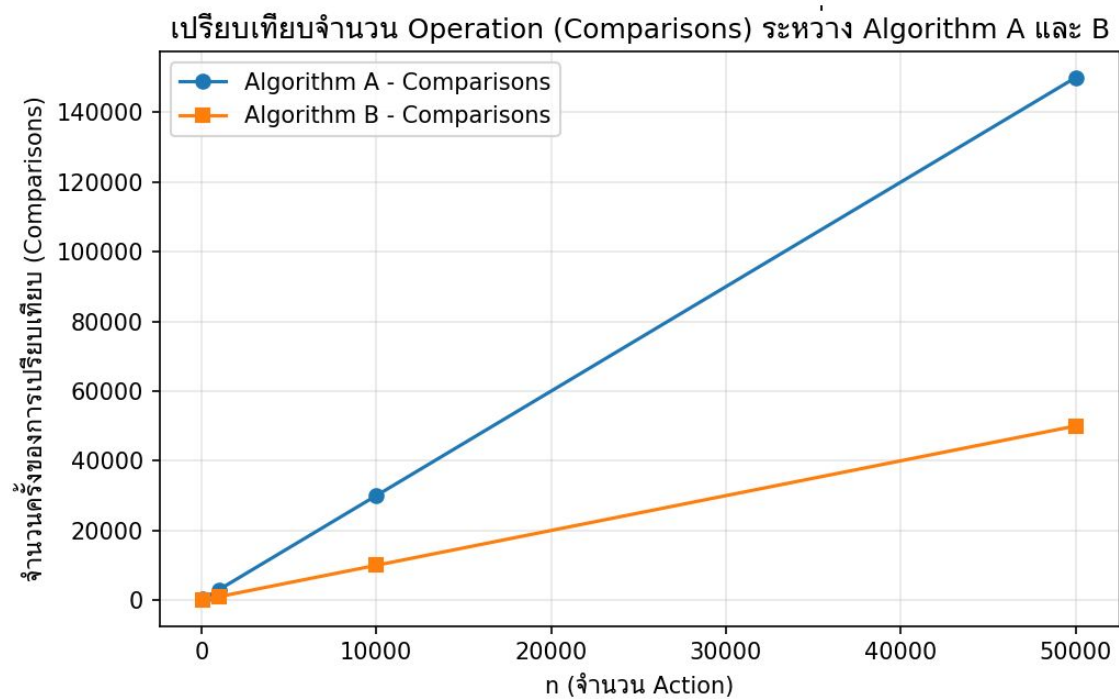
✓ TC08 Undo จน Stack ว่าง

✓ TC09 Redo เมื่อไม่มีรายการ

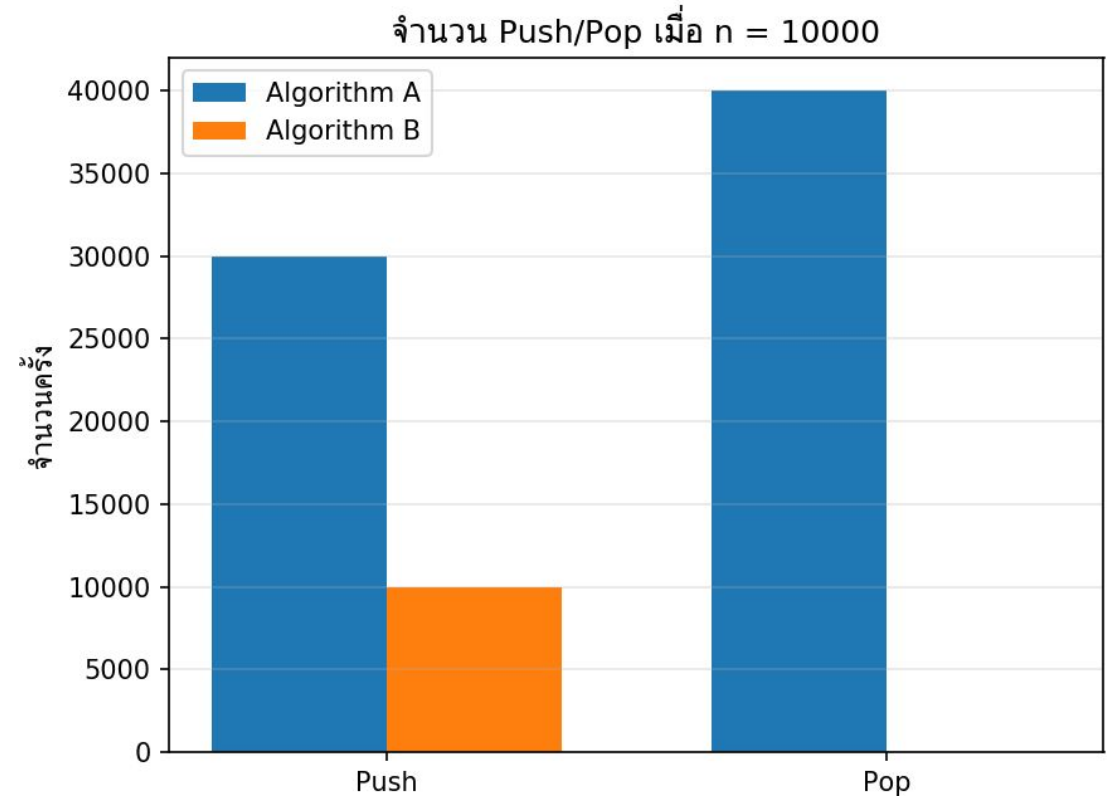
✓ TC10 เพิ่ม Action หลังปิด Case

ผลการทดลอง

ผลการทดลองจริง : $n = 100 / 1,000 / 10,000 / 50,000$ (5 รอบเฉลี่ย)



ที่ $n=10,000$: Algorithm A ทำ Comparisons 30,000 ครั้ง เทียบกับ Algorithm B เพียง 10,000 ครั้ง — ตรงกับที่ทฤษฎีคาดการณ์ ($O(n)$ เทียบ $O(1)$)

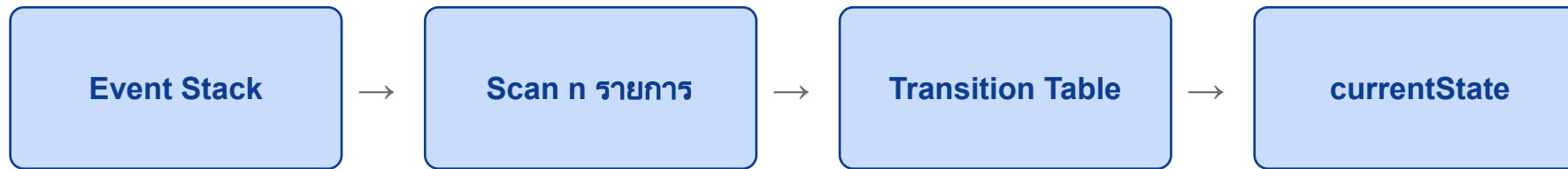


เปรียบเทียบ Algorithm A vs B

หัวข้อ	Algorithm A	Algorithm B
การเก็บ State	คำนวณจาก Stack	เก็บ <code>currentState</code>
ADD	$O(n)$	$O(1)$
Get State	$O(n)$	$O(1)$
Undo / Redo	$O(1)$	$O(1)$
ความซับซ้อนในการดูแล	ง่ายกว่า	ต้องรักษา Invariant
เหมาะกับ	ระบบขนาดเล็ก	ระบบที่มี Event จำนวนมาก

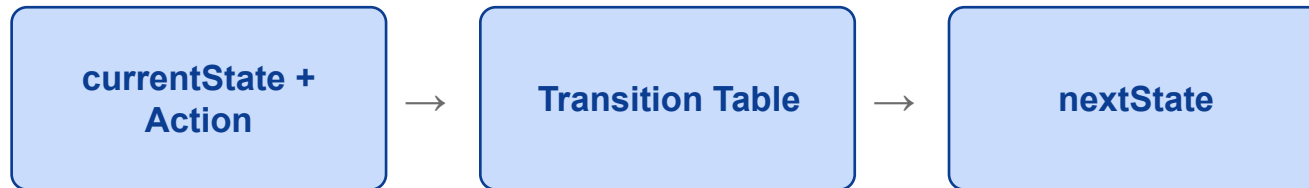
“จุดแตกต่างสำคัญคือ Algorithm A ต้องสแกน Stack เพื่อหาสถานะทุกครั้ง แต่ Algorithm B จำสถานะปัจจุบันไว้ ทำให้การตรวจสอบใช้เวลา $O(1)$ ”

Algorithm A



“ถ้า Event เยอะ → ต้องตรวจสอบเยอะ → $O(n)$ ”

Algorithm B



“ตรวจเฉพาะสถานะปัจจุบัน → $O(1)$ ”

$n = 10,000$
Algorithm A = 30,000 comparisons
Algorithm B = 10,000 comparisons

สรุป

Algorithm B คือทางเลือกที่เหมาะสมกว่าสำหรับการใช้งานจริง

เหตุผล

ระบบต้องอ่าน/ตรวจสอบสถานะบ่อย การมี currentState แบบ $O(1)$ ลดภาระประมวลผลได้มากเมื่อ Action สะสมจำนวนมาก

ข้อจำกัดของงาน

รองรับ Workflow เชิงเส้นเดียว ไม่มีการบันทึกถาวร (In-memory) และรองรับเพียง 1 เคสต่อครั้ง

แนวทางพัฒนาต่อ

บันทึก Event Log ลงฐานข้อมูล, รองรับหลายเคสพร้อมกัน, ปรับปรุงการทดลองด้วย JVM Warm-up